

Lekcja 07: Wąż

Cel

Klasyczny Snake. Nauczysz się używać tablicy jako **listy** o zmiennej długości, przesuwać jej elementy i robić ruch „co którąś klatkę”.

Co nowego

Wąż to ciąg kratek: głowa i segmenty za nią. W tablicach `snake_x[]`, `snake_y[]` element `[0]` to głowa, `[1]` pierwszy segment za nią itd. Zmienna `length` mówi, **ile elementów jest używanych** – tablica ma 200 miejsc, ale wąż na starcie zajmuje 3.

Ruch to przesunięcie każdego segmentu na miejsce poprzedniego, a głowy o jedną kratkę w kierunku ruchu:

```
for (int i = length - 1; i > 0; i--) {    // OD OGONA do głowy!
    snake_x[i] = snake_x[i - 1];
    snake_y[i] = snake_y[i - 1];
}
snake_x[0] = snake_x[0] + dir_x;
```

Dlaczego od ogona? Gdybyś zaczął od `[1] = [0]`, to zaraz potem `[2] = [1]` skopiowałoby już **nową** wartość – cały wąż stałby się głową. Pętla z `i--` (w dół) rozwiązuje problem.

Rośnięcie: nowy segment dopisujesz na końcu (`snake_x[length] = ...`) i zwiększasz `length`. To dokładnie „dodaj element do listy”.

Ruch co N klatek: `if (frame_count() % STEP == 0)` – reszta z dzielenia z lekcji 05. Przy 60 klatkach na sekundę `STEP = 8` daje 7,5 ruchu na sekundę.

Uruchom

Otwórz `gna.cpp`, **Ctrl+Shift+B**. Żółta głowa i dwa zielone segmenty jadą w prawo. Strzałki zmieniają kierunek (nie da się zawrócić o 180°). Czerwona kratka to jedzenie – po zjedzeniu wąż rośnie, wynik w rogu. Wąż **wyjeżdża za ekran** i nic się nie dzieje – to zadanie 2.

Jak to działa

Przeczytaj `frame()`: skręt (`pressed`, bo jedno naciśnięcie = jeden skręt), ruch co `STEP` klatek (przesunięcie, głowa, jedzenie), rysowanie w pętli po wszystkich segmentach. `if (i == 0)` – głowa w innym kolorze.

Zadania

1. ★ Zmień `STEP` na 5 (szybciej) i kolory. Co się dzieje przy `STEP = 1`?

2. ★★ **Ściany.** Dodaj `bool dead = false;` (zeruj w `setup`). Po ruchu głowy: jeśli `snake_x[0] < 0` lub `>= GRID_W`, albo `snake_y[0] < 0` lub `>= GRID_H` – `dead = true`. Gdy `dead`: nie ruszaj węża, napisz „GAME OVER”, po A `restart()`. Dodaj `watch("dead", dead);`. (test 1)
3. ★★ **Własny ogon.** Po ruchu głowy pętla `for (int i = 1; i < length; i++)`: jeśli głowa jest w tej samej kratce co segment `i` – `dead = true`.
4. ★★ Jedzenie nie może pojawić się na węź. W `place_food()` po losowaniu sprawdź pętlą, czy kratka jest wolna; jeśli nie – losuj jeszcze raz. Podpowiedź: pętla `while (!free) { ... }` – „powtarzaj, dopóki” (patrz słowniczek).
5. ★★★ Wąż przyspiesza: `STEP` zamień na zmienną `step`, po każdym jedzeniu `step = max(2, step - 1)`. Wersja alternatywna: zamiast umierać na ścianie, wąż **przechodzi na drugą stronę** – `snake_x[0] = (snake_x[0] + GRID_W) % GRID_W`.

Sprawdź sam

Po zadaniu 2, bez klawiszy, wąż jedzie w prawo od kratki 10 i po 30 ruchach (240 klatek) uderza w ścianę:

```
console_sim.exe --game waz --frames 301 --trace 60
```

W ostatniej linii `dead=true`, a `head_x` zatrzymane na 40.

Dla ciekawych

Snake da się napisać bez przesuwania całej tablicy – wystarczy pamiętać, gdzie jest głowa i ogon, a tablicę traktować jak „kółko” (bufor cykliczny). To ta sama sztuczka, którą używa się w konsoli do buforowania klawiszy.

Słowniczek

po polsku	w C++
ile elementów używam	osobna zmienna, np. <code>length</code>
dodaj na koniec listy	<code>t[length] = v; length++;</code>
pętla w dół	<code>for (int i = n - 1; i > 0; i--)</code>
powtarzaj, dopóki warunek	<code>while (warunek) { ... }</code>
większa z dwóch	<code>max(a, b)</code>